Insights on Microservice Architecture Through the Eyes of Industry Practitioners

Vinicius L. Nogueira
Fernando S. Felizardo
Aline M. M. M. Amaral
Department of Informatics
Maringá State University
Maringá, Brazil

Wesley K. G. Assunção
Department of Computer Science
North Carolina State University
Raleigh, United States of America

Thelma E. Colanzi
Department of Informatics
Maringá State University
Maringá, Brazil

2024 IEEE International Conference on Software Maintenance and Evolution (ICSME) | 979-8-3503-9568-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICSME58944.2024.00080

***Abstract*—The adoption of microservice architecture has seen a considerable upswing in recent years, mainly driven by the need to modernize legacy systems and address their limitations. Legacy systems, typically designed as monolithic applications, often struggle with maintenance, scalability, and deployment inefficiencies. This study investigates the motivations, activities, and challenges associated with migrating from monolithic legacy systems to microservices, aiming to shed light on common practices and challenges from a practitioner's point of view. We conducted a comprehensive study with 53 software practitioners who use microservices, expanding upon previous research by incorporating diverse international perspectives. Our mixed-methods approach includes quantitative and qualitative analyses, focusing on four main aspects: (i) the driving forces behind migration, (ii) the activities to conduct the migration, (iii) strategies for managing data consistency, and (iv) the prevalent challenges. Thus, our results reveal diverse practices and challenges practitioners face when migrating to microservices. Companies are interested in technical benefits, enhancing maintenance, scalability, and deployment processes. Testing in microservice environments remains complex, and extensive monitoring is crucial to managing the dynamic nature of microservices. Database management remains challenging. While most participants prefer decentralized databases for autonomy and scalability, challenges persist in ensuring data consistency. Additionally, many companies leverage modern cloud technologies to mitigate network overhead, showcasing the importance of cloud infrastructure in facilitating efficient microservice communication.**

***Index Terms*—Software Architecture, Service-oriented Architectures, Software Modernization, Industry Practices, Survey**

I. Introduction

Legacy software systems often present challenges in meeting the demands of modern business [1], [2]. Since monolithic architecture had been the norm for decades, most of the existing software systems adopt such an architecture [3]. However, the very nature of large-size monoliths presents several challenges, in which their inherent complexity makes maintenance and evolution cumbersome [4]. For instance, the deployment of monolithic applications tends to be suboptimal, and they also have limited scalability and technological flexibility [4].

To address these problems and take advantage of enticing promises made by the rise of cloud computing [5], companies are turning to microservice architecture as a means to modernize their legacies [2], [6]–[8]. Microservice architecture is an architectural style that emphasizes the splitting of an application into small and lightweight services, which are specifically built to perform a very cohesive business capability [9]. This style is an evolution of traditional service-oriented architecture [10]. Architectures such as microservices carry valuable characteristics that help with the limitations of monolithic systems, such as easier maintenance, improved scalability, and partial deployment [11], and allow users to capitalize on the advantages offered by cloud environments [12].

Despite the benefits of a service-oriented architecture, modernizing a monolith through microservices remains challenging [2], [13]. There is no one-size-fits-all architecture that can be applied universally, and therefore, each microservice architecture must be tailored to meet the specific needs of an application [11], [13]. Several studies explored the migration of legacy systems to microservice architecture [2], [13]–[16]. However, there is still a limited amount of in-depth research on the practical aspects for conducting the modernization.

To gain a practical view of modernization with microservices, Zhou et al. [17] conducted interviews with practitioners from 20 companies to investigate the practical implications of microservices architecture. Their findings show the discrepancies between idealized visions and real-world practices concerning this architectural style, and emphasize the importance of informed decision-making and the need to proactively address potential challenges in adopting microservices. Other papers also present surveys with practitioners [7], [18], [19]. However, while these papers explore strategies and challenges for migration, they do not provide in-depth insights into how practitioners deal with certain issues. These studies often focus on a single approach, some using quantitative data [19], and others are qualitative studies [7], [20].

Even though existing studies indicate organizational challenges, including mindset shifts towards agile processes and team collaboration, they do not fully address the entire modernization process. For instance, little is known about data model transformation and the integration of microservices within existing infrastructure. These gaps highlight the need for further research combining qualitative and quantitative analyses to provide a more comprehensive understanding of microservices migration in practice. In a previous study,

2576-3148/24/$31.00 ©2024 IEEE
DOI 10.1109/ICSME58944.2024.00080

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

we conducted a preliminary survey with 56 practitioners, primarily from Brazil, of whom 35 use microservices [21]. However, we predominantly evaluated the technical aspects of the modernization process and investigated (in a *confirmatory* study) whether industry practices were aligned with academic principles based on the roadmap presented by Wolfart et al. [13], ultimately confirming this alignment.

Motivated by the above-mentioned limitations, we investigate a different perspective from our previous work. In this study, we adopt a more *exploratory* approach to uncover how professionals navigate the complexities of microservice migration in their organizations. In addition to the common technical, organizational, and operational perspectives of microservice migration, we take a more practical approach to investigate how and why practitioners deal with challenges related to data consistency and common pitfalls to avoid during migration. Other limitations of existing studies is that they usually have a few dozens (around 20) of participants [7], [18], [19] or are geographically limited to a single region [18]. To diversify our sample and obtain more heterogeneous results, our survey includes more participants from different regions. We increased the total number of microservice users from 35 to 53, with 18 new responses. Using this newly collected data, we explore the motivations, activities, and challenges of migrating from monolithic legacy systems to microservices.

More specifically, our study consists of quantitative and qualitative analyses of data gathered from a survey-based study involving software practitioners. We essentially want to better understand *why* and *how* practitioners make certain decisions when modernizing their systems. We investigate four key aspects of the legacy system modernization process: (i) the driving forces compelling companies to migrate, (ii) the methodologies employed by these companies in their modernization process, (iii) the strategies implemented to handle database and data consistency concerns, and (iv) the prevalent challenges encountered throughout the modernization.

The results reveal diverse motivations for migration, such as the need for improved scalability and maintainability, confirming previous findings [7], [18], [22]. We identified a range of methodologies and strategies practitioners employ, particularly in managing data consistency and ensuring efficient deployment. To ensure data consistency, sophisticated techniques such as Command Query Responsibility Segregation (CQRS) and Event Sourcing were mentioned when working with decentralized systems. Additionally, efficient deployment is often achieved through robust monitoring tools, automatic monitoring mechanisms, and leveraging modern cloud technologies to mitigate network overhead and guarantee smooth operation across microservices. Furthermore, we discuss some challenges, including the complexity of testing microservice environments and the ongoing debate between centralized and decentralized database management approaches. The findings reaffirm the critical role of cloud infrastructure in facilitating effective microservice communication and indicate areas where further research and tailored solutions are needed.

The main contributions of our work include providing a comprehensive view of the current practices and challenges industry that practitioners face when migrating to microservices. The insights gained from this study can help developers and companies make informed and cost-efficient decisions when considering microservice adoption. By documenting real-world experiences and identifying common obstacles, our research offers valuable guidance for overcoming the hurdles associated with microservice implementation. Moreover, the study lays the groundwork for future research, particularly in areas such as testing methodologies and database management in microservice architectures, thereby supporting the continued evolution of best practices in this domain.

The remainder of this paper is structured as follows. Section II reviews related work. Section III details the study design and execution, with our research methodology and data collection process. Section IV presents the results and discussion. Section V contains extracted practical lessons for practitioners. Section VI addresses the threats to the validity of our study and Section VII concludes the paper by summarizing the main findings and suggesting directions for future research.

II. Related Work

Wolfart et al. [13] present a comprehensive roadmap for modernizing monolithic legacy systems using microservices. The proposed roadmap comprises eight activities grouped into four phases: initiation, planning, execution, and monitoring. To understand how monolithic legacy systems are migrated to microservices, the authors used a systematic mapping method based on 62 studies to extract information regarding the motivations to start a modernization, the activities used to conduct the modernization, and the information used as input and generated output for each of those activities. The main contribution of their study includes providing a roadmap for practitioners to plan, execute, and monitor the modernization process, serving as a reference for researchers to design new studies, and motivating tool builders to address existing needs.

Using Wolfart et al.'s roadmap, in a previous study, we investigated whether industry and academia are aligned in their approaches to modernizing legacy systems using microservices [21]. We surveyed 35 software practitioners who use microservices, and the results indicate that, for the most part, companies follow best practices shown in the literature, performing activities outlined in the modernization roadmap [13]. However, the responses show that data persistence is still controversial among practitioners, with many companies using decentralized databases and managing data consistency through application logic. Additionally, in our prior work, the analyses were focused on technical aspects, such as the integration and communication tools the participants used. Overall, that study concludes that industry and academia are generally well-aligned. However, some areas in the context of microservices still needed further research. For instance, our survey did not include questions regarding the process of understanding the legacy system before migration, even though most participants said they performed this activity.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

More recently, Zhou et al. [17] conducted an empirical study to explore the divergences between the ideal concepts of microservices architecture and real industry practices. Through interviews with practitioners from 20 software companies, the study identified common practices and challenges in implementing microservices. They synthesized their findings into eight pairs of practices and pains and extracted five critical decision-making aspects to help balance the benefits and drawbacks of the architecture. The authors conclude that (i) the benefits of microservices are confirmed by practitioners; and (ii) the potential problems must be carefully addressed, including systematic evaluation, organizational transformation, decomposition, distributed monitoring, and bug localization, to optimize the implementation of microservice architectures.

Di Francesco et al. [19] surveyed 18 practitioners to gather quantitative information about migrations towards microservices architecture. Their results emphasize the importance of well-planned strategies and effective team collaboration. Among the challenges identified, the authors reinforce the migration of pre-existing data from legacy databases to microservices as a significant open problem. Sixty percent of their participants said they maintained their original database structures even after migrating. Similarly, Fritzsch et al. [18] conducted interviews with 16 professionals from 10 German-based companies, focusing on intentions, strategies, and challenges. Both of these studies reveal significant organizational challenges. However, these studies provide either qualitative or quantitative insights but leave open questions regarding the approaches practitioners use to tackle challenges.

Our study addresses these gaps by incorporating qualitative and quantitative analyses from a larger and more diverse sample of 53 practitioners from various international backgrounds. We report on real-world experiences associated with migrating to microservices, covering the entire process from the initial motivations for migration through the migration itself to the strategies practitioners employ for dealing with challenges such as maintaining data consistency and ensuring efficient deployment. By doing so, our research provides a more holistic understanding of the practical realities of microservices adoption and offers actionable insights for professionals.

III. Study Design and Execution

This section presents a detailed description of our study.

A. Goal and Research Questions

The main goal of our study is to *understand **why** and **how** industry practitioners make their decisions when migrating monolithic legacy systems*. This understanding can help other professionals make informed decisions when performing similar activities. To this end, we used a multi-method approach, performing quantitative and qualitative data analyses.

When talking about microservice migration, existing work usually focuses on technical [17], [21], economic [7] or even organizational [17]–[19] aspects of the migration process, but more importantly, these researches predominantly focus on identifying what goes into these concepts. In contrast, we aim at a more practical perspective, leveraging the real-life experiences of practitioners to go further and understand how and why industry professionals do or do not make specific decisions when modernizing legacy systems with microservices.

We translated our main research goal into four research questions (RQ), as follows:

RQ1. Why do companies migrate monolithic legacy systems to microservices? By answering this research question, we wanted to understand why practitioners/companies migrated to microservice architectures and the motivations driving companies to adopt microservices. This can help practitioners assess and align the potential benefits with their organizational goals.

RQ2. How do practitioners approach the migration process? We aimed to understand participants' strategies to execute the migration successfully. This question seeks to identify and characterize the activities employed during the migration process, establishing a basis for the migration.

RQ3. What are the concerns regarding data persistence in modernizing legacy systems with microservices? We address some of the pressing issues regarding the recurrent topic of data management in migration. Addressing this question will detach critical issues related to this topic.

RQ4. What are the main challenges and pitfalls to avoid when migrating legacy systems to microservices? We studied the common challenges and pitfalls faced by practitioners. By answering this question, we aim to equip professionals with knowledge to anticipate and mitigate similar problems, leading to a more successful migration.

B. Design of the Questionnaire

In this study, we performed survey-based research to observe the modernization processes from the perspective of software development companies' practitioners. For that, we designed a questionnaire composed of 29 questions, of which 20 are close-ended, and 9 are open-ended questions. The open-ended questions were included so that participants could explain their answers in detail. Due to space limitations, the questionnaire is available in our supplementary material [23]. We used a Google Form[1] to collect practitioners' responses.

Based on the migration process, the questions are grouped into subjects of interest (or categories). These subjects are: (i) Driving forces for migration, question Q1; (ii) Execution of the migration process, questions Q2 to Q10; (iii) Database concerns, questions Q11 to Q21; and (iv) Challenges and difficulties, questions Q22 to Q29. These four categories are designed to help answer a corresponding research question. Additionally, the questionnaire has questions used to characterize the participants (i.e., demographic questions).

In our prior survey [21], we noted that many companies perform the activity of "Understanding the legacy system" during modernization. However, since the original questionnaire did not have in-depth questions on this topic, two new questions

[1] https://www.google.com/forms/about/

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

were added regarding how companies understood their original system before modernization. These two questions only have 18 responses from the new participants, as explained below.

The questionnaire of our preliminary study [21] received 56 responses, with 35 participants using microservice architecture, which was considered in our study. This present work expanded the survey to include participants from multiple countries worldwide, adding 27 new responses. Of these, 18 were from microservice users, increasing our original sample size by more than 50%. Hence, our present study has a set of 53 responses from microservice users, used as a source of information to answer the RQs.

C. Search and Characterization of Participants

To find participants, we first looked for researchers who published papers related to microservices architecture; however, this strategy resulted in low participation. As a second approach, we posted on different social media websites (LinkedIn, X—former Twitter, and Facebook) and contacted students and former colleagues from different countries. This strategy proved much more successful, reaching more practitioners and enabling us to collect enough answers to our questionnaire.

Adding our previous data to the newly collected replies, we now have a sample of 53 responses. Among the geographic distribution of the participants: 41 (77.4%) of those responses are from Brazil, and 12 (22.6%) from different countries, spread as follows: Ireland 2 (3.8%), England 2 (3.8%), Spain 2 (3.8%), Uruguay 2 (3.8%), Argentina 1 (1.9%), Australia 1 (1.9%), Portugal 1 (1.2%) and United States of America (USA) 1 (1.2%) response.

Most participants are Software Engineers, namely 25 (47.2%). Other positions include Software Architect 8 (15.1%), Project Manager 7 (13.2%), Software Analyst 5 (9.4%), Researcher 2 (3.8%), Software Tester 2 (3.8%), Software Specialist 2 (3.8%), CEO 1 (1.9%), and Innovation Director 1 (1.9%). The majority (29 or 54.7%) of the companies where participants work defines their hierarchical structure considering the business area. At the same time, 18 (34%) use the development area, and the remaining 6 (11.3%) use a mixed structure of business and development.

Our participants bring diverse experiences to this study. Regarding overall experience with software development, 24 (45%) participants have between 4 to 10 years of experience, while 10 (19%) have 11 to 15 years of experience. A smaller group of 7 (13%) participants has 16 to 20 years of experience, and another 7 (13%) have been in the field for over 20 years. Notably, 27 (51%) of the participants have been with their current company for 3 years or less, 15 (28%) have worked there for 4 to 8 years, and 2 (4%) have been in the same company for over 15 years. Regarding experience with microservices architecture migration, 26 (49%) of the participants reported having some experience (5 years or less), while 8 (15%) are considered experienced users with 5 to 8 years of experience. However, 6 (11%) participants have no experience in MSA migration. As these questions were not mandatory, some participants did not respond. Table I illustrates the total responses.

TABLE I
EXPERIENCE DEMOGRAPHICS

Experience	Number (%)
Overall experience with software development	
4 to 10 years experience	24 (45%)
11 to 15 years experience	10 (19%)
16 to 20 years experience	7 (13%)
More than 20 years experience	7 (13%)
No response	5 (9%)
Time working with current company	
3 years or less	27 (51%)
4 to 8 years	15 (28%)
9 to 15 years	4 (8%)
More than 15 years	2 (4%)
No response	5 (9%)
Experience with MSA migration	
No experience	6 (11%)
5 years or less of experience (Some experience)	26 (49%)
5 to 8 years experience (Experienced user)	8 (15%)
No response	13 (25%)

D. Data Analysis

We analyzed the responses in each category to answer the proposed research questions. For the close-ended questions, we performed a quantitative analysis. For the open-ended ones, we performed a thematic synthesis following the recommendations presented by Cruzes & Tore [24].

IV. RESULTS AND DISCUSSION

The results are presented in four subsections, one for each category, and these subsections show our extracted quantitative and qualitative data and answer their respective research question.

A. RQ1 - Driving Forces for Migration

According to the literature [13], the modernization of legacy systems with microservices is driven by various motivating factors. However, to understand the driving forces that lead companies to migrate their systems in practice, we asked participants (a multiple-choice question) *why* they decided to modernize legacy systems through microservice architectures.

Table II presents the selected driving forces ranked by popularity. The results indicate a slight preference towards technical and operational motivations, although participants considered all perspectives. The responses reflect a balanced interest in the benefits microservices offer. Specifically, the top three driving forces identified are consistent with those mentioned in existing literature [13], [18]: *Easier maintenance and evolution* (81.1%), *Optimized scalability* (79.2%), and *Independent and automated deploy* (75.4%). These driving forces were cited by more than 70% of participants, indicating their importance in the decision to migrate.

Other significant motivations include *Technology flexibility* (54.7%) and *Reduced time to market* (47.1%). These factors suggest that the ability to adapt and innovate rapidly is a

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DRIVING FORCES

Driving Force	Total (%)	Perspective
Easier maintenance and evolution	43 (81.1%)	Technical
Optimized scalability	42 (79.2%)	Operational
Independent and automated deploy	40 (75.4%)	Operational
Technology flexibility	29 (54.7%)	Technical
Team Autonomy	26 (49.1%)	Operational
Loosely coupled services	25 (47.1%)	Technical
Reduced time to market	25 (47.1%)	Organizational
Agility enabler	18 (33.96%)	Organizational
Cohesive services	17 (32.1%)	Technical
Infrastructure facilities	14 (26.4%)	Operational
Easier reuse	12 (22.7%)	Technical
Domain-driven design delimited contexts	1 (1.9%)	Technical
Better utilization of cloud computing resources	1 (1.9%)	Technical
Better operational autonomy	1 (1.9%)	Operational

common consideration for organizations. Additionally, *Team Autonomy* (49.1%) and *Loosely coupled services* (47.1%) point towards a desire for greater independence and modularity within development teams.

Less commonly cited driving forces include *Agility enabler* (33.9%), *Cohesive services* (32.1%), *Infrastructure facilities* (26.4%), and *Simpler Reuse* (22.6%). Also, few participants used the "others" option to suggest their particular driving forces, naming unique reasons such as *better operational autonomy*, *better utilization of cloud computing resources*, and *domain-driven design delimited contexts* (cited by only 1 participant each, 1.9%).

Answering RQ1: The primary driving forces for adopting microservices are predominantly operational and technical, focusing on enhancing maintenance, scalability, and deployment processes. This is in accord with the broader industry trends [7], [14], [17], which emphasize the technical and operational advantages of microservices in software development, such as easier maintenance/evolution and the possibility of using multiple technologies [25], [26].

B. RQ2 - Performing the Modernization

To understand *how* companies carry out the migration process, we asked five multiple-choice and three open-ended questions. First, participants were asked about their preferred activities or strategies to understand their legacy systems before migration. This is one of the newly introduced questions.

The most common strategy chosen was *A study with the most experienced people that worked in the development of the legacy system*, selected by 10 out of 18 (55.6%) participants. This result aligns with the findings by Taibi et al. [7], suggesting that involving experienced developers can significantly reduce the effort needed to develop microservice systems. *Domain-Driven Analysis* (50%) and *Reading system documentation* (44%) were also frequently cited. These activities are well-regarded in the literature for their effectiveness in software development. Domain-driven design is a very flexible methodology [27], crucial for defining the boundaries and responsibilities of each created microservice [28], while thorough documentation provides additional information in any project [29]. *Dynamic Analysis* (39%) is a fairly mentioned strategy, reflecting a need to understand runtime behavior, which can be a concern during modernization [30].

We also asked an open-ended question regarding the activities performed to understand the legacy system, and using thematic analysis to analyze our 18 responses, we arrived at four major themes for this subject.

The first theme, **Understanding Through Decoupling and Monitoring**, illustrates that participants perform the modernization of legacy systems through strategies such as decoupling components and thorough monitoring. Participant P67 reinforced the importance of breaking down the system into manageable parts and connecting them to new microservices while maintaining the legacy system until stability is ensured: "*As far as I'm aware, the modernization occurs by understanding the legacy system into components that could be decoupled [...] we execute the modernization, and slowly connect consumers to the new microservices while still having the legacy. Once they are stable, the legacy system can be decommissioned.*"

As a second theme, **Importance of Documentation** also plays an important role, as emphasized by P68: "*The development team had several meetings with the legacy application team, in which the code was presented, and many UML diagrams were specified to document existing legacy application and support the understanding.*" The emphasis on documentation ensures that there is a clear and shared understanding of the system's structure and behavior.

Two other themes are **Domain-Driven Design (DDD)** and **Effective Collaboration and Communication**, both mentioned by participants as key concepts in understanding the legacy system. Participants P69, P75, and P82 pointed out the value of DDD in comprehending business complexities and planning migration. The response of P69 addresses both of those themes: "*DDD techniques were very useful to understand the business complexities and core domain. To plan and execute the migration, activities mainly involved keeping close contact with the development/maintenance teams [...].*" Team meetings and advice given by more experienced practitioners were also cited by participants P73, P76, and P78, who said: "*[...] There are experts in the company that help us understand and translate system architecture.*"

We also asked practitioners about their strategies to decompose their legacy systems into microservices. The results indicate that the most prevalent approach is *decomposing the system by business capabilities, defining the responsibility of each microservice*, chosen by 28 (52.8%) participants. This approach aligns with report studies on monolith refactoring [31], which advocate for organizing systems by business capabilities. Some papers suggest that each microservice should have one and only one business responsibility [28]. An *incremental strategy, focusing on one microservice at a time*, was selected by 27 (50.9%) participants. This method often involves applying patterns like the strangler fig pattern [18], allowing for the gradual replacement of monolithic components with microservices. Less commonly, some companies decomposed

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

their systems *based on system operations and their data* (20.8%), *using UML diagrams* (3.8%), or *specialized tools like service cutters* (1.9%) to aid in decomposition.

Participants were then asked about the criteria used to decide which microservices to extract. The results showed *scalability* was considered the most relevant criterion, with 40 (75.5%) participants noting its importance. This is likely because enhancing scalability allows for the expansion of microservices without requiring modifications to other system parts [32]. As Newman discusses [33], breaking processing into individual microservices enables independent scaling, allowing companies to cost-effectively scale only those parts that constrain load while scaling down or turning off less utilized microservices when not needed. *Requirements* (52.8%) and other criteria related to modularity such as *reuse* (43.4%), *cohesion* (37.7%), and *coupling* (35.8%), were all notably mentioned. This indicates an interest in maintaining well-defined and loosely coupled microservices. These findings are supported by various studies [14], [25]. The possibility of lower coupling and higher cohesion on microservice-based systems is also noted several times in literature [5], [14], [34].

For the analysis regarding the process of migrating their systems to microservices, among the 53 participants, 27 (50.9%) opted to gradually migrate a system initially implemented in a different architecture to the microservice architecture. Conversely, 5 (9.4%) participants decided to migrate immediately, while 18 (34%) participants built their new systems using the microservice architecture from scratch. The prevalence of gradual migration is consistent with studies such as the work by Fan & Ma [35], in which the authors justify gradual migration to maintain system stability and solve problems as they come up and are identified during the migration process. For the companies who chose microservices from scratch, this can be attributed to a need for faster development cycles [36], [37]. Although developing a system using microservice architecture can have higher initial costs, the potentially poor quality of the legacy code makes understanding and reusing the implementation a laborious activity [38]. Thus, practitioners mainly rely on the team's knowledge of the legacy systems instead of reusing legacy code.

To investigate the activities involved in migrating from monolithic architectures to microservices, we asked participants about the specific steps taken during their migration process. The most frequently cited steps were *decomposing the legacy system and identifying microservices* (75.5%) and *understanding the legacy system* (71.7%). This is supported by recommendations found in modernization literature [13], where understanding and decomposing the legacy system is foundational to the process [7], [28]. These steps ensure that the migration is based first on comprehending the existing system and its functionalities. *Defining the microservice architecture base* (58.5%), *Monitoring microservices and infrastructure* (58.5%), *Assessing the criteria/need that will lead to migration* (54.7%), and *Verifying and validating microservices* (50.9%) were all selected by over 50% of companies. These activities are also well-documented in academic literature [13], [17] and indicate that, for the most part, companies are executing methodologies proposed by experts.

In an open-ended question, we asked participants about their concerns regarding microservices granularity. This question had 43 responses, which resulted in five major themes. The first theme is **Defining Clear Responsibilities**. This emphasizes the importance of establishing clear boundaries for each microservice to avoid duplicated functionalities and ensure each service has a specific purpose. Participants mentioned the need for microservices to have well-defined responsibilities, such as P8's focus on reducing coupling and ensuring clear service responsibilities: "*The main concern is defining the responsibility of each microservice and reducing coupling between them.*" Participant P20 also said: "*Each microservice should perform one task and one task only.*" This theme stresses the need for purposeful microservices that handle specific tasks, enhancing the overall efficiency and maintainability of the system.

The theme **Management/Integration and Communication Complexity** addresses the challenges of managing and integrating numerous microservices. Participants like P5 and P77 pointed out the difficulty in managing highly granular services, while P30 and P73 underscored the complexities in integrating these services, especially with multiple REST calls and communication issues. The increased complexity in managing a microservice ecosystem, as noted by P21 and P81, can lead to significant integration errors and communication problems, complicating the overall system architecture and maintenance. For instance, P82 noted: "*Problems with communication between the microservices, as more services exist, more complex is the communication between them.*"

The third theme, **Quality Attributes**, encompasses concerns about cohesion, reusability, and other quality attributes of software architecture. Participants mentioned the balance between reusability and performance, with P4 noting: "*The fundamental challenge is finding an adequate level to ensure reusability without compromising performance.*" Network overhead and horizontal scalability were also important considerations, as discussed by P69 and P7, respectively. Concerns with Maintenance and load balancing can be seen in the responses of P38, "*Code maintenance/database structure, load balancing*" and P74, "*High granularity can increase complexity, resulting in harder maintenance.* ". This shows the importance of ensuring the system remains efficient and manageable over time.

For the theme **Balancing Service Size**, we see the attention to avoiding extremes in microservice granularity. Participants such as P15 and P28 warned against creating "nanoservices" that are too small and difficult to manage, as well as avoiding distributed monoliths that are too large and complex. P15 said: "*We strive not to create nanoservices or distributed monoliths; we aim to create services focused on a business domain.*" This same concept can be seen in this quote from participant P70: "*The concerns are to not implement too small microservices, which would lead to creating a huge number of services, and do not implement too large microservice, which will be considered a monolith.*"

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

The fifth theme, **Team and Resource Management**, encompasses the challenges of managing the human and resource aspects of microservice architecture. P25 highlighted the need for a large team to manage numerous microservices. Similarly, P11 discussed the need to keep microservice scopes manageable for small and multifunctional teams: "*At most, they [services] should have a scope that a cross-functional team of 5-8 can autonomously manage.*" Effective team and resource management are critical for maintaining and scaling the microservice architecture appropriately without overwhelming development and operations teams.

For the last open question regarding the migration process, we asked participants whether their companies followed a defined process for defining new microservices. Our thematic analysis found four major themes in 42 responses.

The first theme concerns **Ad Hoc and Informal Processes**. Many participants reported a lack of a formal process for defining new microservices, relying instead on ad hoc and informal methods. For example, P73 said: "*It's like an ad hoc process; we do whiteboard sessions to discuss the team ideas and define a structure for the new microservice if it does not fit in an existing one.*" In total, 19 participants explicitly stated there was no formal process in place. P67 and P68 mentioned that while there was no formal process, engineers would propose system changes to management and periodically assess microservice quality.

Next, **Architecture Team's Role** emphasizes the involvement of architecture teams in the microservice definition process. Participants like P2 noted that the architecture team analyzes requests from the product team to determine whether a new microservice is needed, or if an existing one can be enhanced: "*There is an architecture team that analyzes the request from the product team for a new functionality and evaluates whether it is necessary to create a new microservice or if this new function can/should be included in an existing one.*" Yet, P43 mentioned that architecture evolution often leads to the extraction of microservices from existing monoliths.

The third theme, **Business Context and Needs**, connects microservice creation to business requirements. Participants shared that new microservices are often defined based on business contexts and needs. P38 mentioned that a well-defined business context is a strong indicator for implementing a new service. Similarly, P80 said: "*They are generated as the business needs.*" Participant P81 gave three situations where his company usually creates new services: "*1) a new domain object is being introduced; 2) an existing service became too big, so it's being split out; 3) a new 'action' (i.e., a set of functionalities) is being introduced in the system, and it doesn't logically fit into an existing service.*"

Lastly, we have **Other Methods** that includes various less-mentioned considerations, such as periodic quality assessments (P68) or monitoring production behavior and creating documentation (P72). Sprint review meetings (P3) and mapping and prioritizing manual processes (P16) were also mentioned as part of the microservice definition process.

Answering RQ2: Companies adopt a systematic and incremental approach to migrating monolithic legacy systems to microservices. The most common strategy used by 50.9% of participants is *decomposing systems by business capabilities*. Scalability is the most critical criterion for defining microservice candidates, cited by 75.5% of participants. Other significant criteria include requirements, reusability, cohesion, and coupling. Practical strategies for understanding legacy systems, such as involving experienced developers [7] and domain-driven analysis [27], [28], are widely used and supported by the literature.

C. RQ3 - Database and Data Consistency Concerns

Managing data while ensuring consistency is a recurrent challenge when migrating legacy systems to microservices, mainly because of the distributed nature of this type of architecture [33], [39]. Nonetheless, existing work on modernization with legacy systems falls short when the topic is data management. Thus, we asked participants several questions to understand how they handled database structures, transactions, and consistency in their microservice architectures.

First, we inquired practitioners about the structure of the databases used in their microservice architectures. The majority, 31 (58%), reported using a database dedicated to each microservice. Interestingly, this result slightly differs from a 2018 survey with practitioners [19], in which more than half of participants mentioned using mainly one single database for all microservices. Having one database per microservice works well with the principles of microservice autonomy, reducing coupling and enhancing scalability [33], as recommended by Lewis & Fowler in their seminal article about microservices [9]. However, decentralized data leads to the problem of guaranteeing the consistency of said data [40]. Despite this recommendation, 10 (19%) participants indicated using a centralized database accessed by all microservices. While convenient, this design decision can increase coupling and reduce the autonomy of microservices [33]. Interestingly, while participants confirm using centralized databases in microservices to address data consistency issues [41], these participants acknowledge that a certain level of centralization is counterproductive to the spirit of the architecture [36], [41]. *Connection pooling*, a technique to improve database connection efficiency [42], was used by 40 (75%) participants, reflecting the interest in maintaining the performance of microservice-based applications.

When asked whether their choice of microservices was influenced by the transactions they execute in the database, 25 (47%) participants indicated that transactions were evaluated and refactored when necessary to allow for separation into microservices. Conversely, 10 (19%) did not evaluate transactions, and 6 (11%) evaluated but did not refactor them.

Regarding consistency strategies, 22 (42%) practitioners mentioned they adopted application-controlled eventual consistency, while 16 (30%) used atomic transactions in the

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

database. The choice between these approaches reflects trade-offs between consistency and performance [43]. A few companies, 5 (9.4%), use both approaches case-to-case.

Handling database tables across multiple microservices is another point of interest, as it can lead to tightly coupled services. In our survey, 18 (34%) participants reported that their microservice-based systems involve multiple services managing the same tables. According to Ntentos et al. [44], it is crucial to determine the data to be shared between microservices, the frequency of updates, and the primary data provider. These decisions ensure that data sharing does not result in tightly coupled services. Interestingly, 36 (68%) participants stated that their microservices do not share persistent business classes with each other, which supports the autonomy and decoupling principles of microservice architectures [36].

To the participants who said their microservices *do* share persistent business classes, we asked an open question about what strategy was adopted to share classes of persistent entities used by more than one microservice. Our analysis of those responses resulted in two major themes.

The first theme, **Common Code Packages and Libraries**, addresses using shared packages and libraries to manage persistent entities across microservices. Participants like P9 and P12 mentioned using a "common package" and a shared Maven library, respectively, while others, such as P13, reported having a shared base project containing all entities: "*We have a base project shared between the microservices, this project contains all entities, even those used by only one microservice.*" This theme highlights the importance of maintaining consistency and reducing redundancy in data handling across services. The theme of **Event Sourcing** comprises the utilization of event sourcing, as cited by P25, as a strategy to manage shared persistent entities.

The performance and scalability of databases were considered important by 31 (58%) participants during the design of their microservice architectures. These findings reveal that companies actively address data persistence and consistency challenges while migrating to microservices. Although guidelines and practices are seen in the literature [33], [45], persistence strategies vary. Issues like data consistency [40], [41] and the complexity of managing multiple databases [46] still pose challenges.

Our questionnaire included an open-ended question directed to participants who considered the performance and scalability of their databases important. We asked those participants about the strategies adopted to scale the database in the face of performance bottlenecks. Our analysis of those responses resulted in five themes.

The first theme is **Asynchronous Processing and Event Handling**, which includes strategies like CQRS (Command Query Responsibility Segregation), mentioned by participants P25 and P39, and asynchronous processing, noted by Participant P5. These methods help manage load and improve performance by separating read and write operations [47]. The second theme, **Testing and Monitoring**, includes load testing and continuous monitoring, as highlighted by P10, P28, and P72. These practices ensure the system's scalability and performance are regularly assessed and optimized. The third theme is **Scalable Database Services**, which refers to using database services that inherently provide scalability, as pointed out by P15: "*Using DB services that already provide scalability.*". The quote by P28, in special, conveys both the first and second themes well, offering an interesting insight into their company's approach: "*We opted to use a Database as a Service (DBaaS) with constant monitoring, from time to time we perform load tests in the systems to evaluate possible code or infrastructural adjustments.*"

The theme **Auto-scaling and Vertical/Horizontal Scaling**, covers strategies like auto-scaling (P19, P67, and P78), vertical scaling (P43 and P69), and horizontal scaling (P66 and P69). These methods allow the system to adjust resources dynamically based on demand. P69 mentioned a case-to-case approach for different types of customers: "*Although rare, bottlenecks can occur if the same customer (tenant) heavily uses different microservices that access the same DB. In the case of a recurrent tenant with such a use case, horizontally scaling the DB (Clusters, Data Replication, Sharing) might be fruitful in the long term. Otherwise, short-term, more simple vertical scaling of the DB servers.*"

Lastly, our fifth theme is **Code Optimization**, which refers to the need for code optimization to improve performance, as mentioned by the participants P68 and P70.

In the final open-ended question regarding database concerns, we asked participants who use centralized databases in their microservice-based systems what factors hindered their decision to partition the database to isolate the persistence of each microservice. This question had 19 responses, resulting in five major themes.

Early Stage Migration Priorities indicates that maintaining a centralized database is often a temporary measure during migration. Participants like P27 noted that they were in the early stages of migration and still using what they called a "distributed monolith." P74 stated: "*Database will be partitioned eventually. It was [not partitioned] yet due to higher priority aspects of the architecture.*"

The second database concerns theme, **Proprietary and Legacy Dependencies**, encompasses challenges related to proprietary databases and legacy system dependencies. P5 and P8 mentioned being constrained by these factors. P8 reported: "*The database is shared by the legacy, and in particular by our BI [business intelligence].*"

We also observe financial constraints as considerable barriers in quotes leading to the third theme **Cost and Licensing Issues**. Participants like P12 and P19 mentioned the high licensing costs associated with new database instances.

In the fourth theme, **Context Separation and Complexity**, the feedback from P39 and P43 provides some interesting information covering the complexities of separating contexts within a monolithic database. P39 shared their approach to handling access to a centralized database: "*[...] access to a single database generally leads developers to choose the easier path, as the desired data can be 'just a join away.' What*

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

I recommend for projects with good prospects for exponential growth is to segregate access into different users and schemas, minimally separating the domains and responsibilities of the application. This adds some complexity, but nothing compared to having another database, and it helps you keep responsibilities segregated."

The last theme found in our analysis is **Data Compliance, Consistency, and Integrity**. P43's answer is noteworthy as it addresses concerns of context separation and data compliance: "*The issue was the design of these microservices without considering what we now regard as best practices. Over time, the applications become complex and consequently difficult to segregate. Another challenge is maintaining consistency between these services.*" P69 and P77 also mentioned the importance of maintaining and ensuring compliance when dealing with sensitive data.

Answering RQ3: The main concerns regarding data persistence focus on database structure, consistency, and performance. Most participants (58%) use a dedicated database for each microservice. Consistency strategies vary, with 42% adopting eventual consistency controlled by the application and 30% using atomic transactions, highlighting trade-offs between consistency and performance. Regarding data sharing, 34% of participants report multiple microservices managing the same table. While 68% avoid sharing persistent business classes. Practical strategies for scaling databases include asynchronous processing, event handling, load testing, continuous monitoring, scalable database services, and autoscaling. Cost, legacy dependencies, and complexity are major barriers to database partitioning.

D. RQ4 - Challenges of Working with Microservices

The migration from legacy systems to microservice architectures presents several challenges [46], [48]. In this section, we explore the prevalent challenges encountered by companies during this modernization process.

The first question to practitioners was about reactivating or reusing legacy code. Among the answers, 10 (19%) participants stated that their companies had to reactivate or reuse some functionality from the legacy system. We then posed an open-ended question, asking participants to detail why they had to reactivate these functionalities. Our analysis of their responses identified three major themes.

The first theme is **Technical Issues in New Implementations**, such as bugs (P10), performance issues (P20), and integration problems (P26, P30, and P80). These issues often necessitate falling back to legacy systems. The second theme is **Strategy for Coexistence and Validation Periods**, for which P75 reported that their systems are designed to allow legacy and microservice architectures to coexist: "*What we do is design our system to allow both legacy and microservices to co-exist for a certain time. Once the solution is validated, the feature on the legacy system is shut down.*" This approach is aligned with recommendations from [33].

For **Meeting Customer-Specific Needs**, participants like P26 and P83 mentioned that legacy functionality is sometimes retained to meet specific customer requirements during the transition. For instance, P83 stated: "*The legacy system was customized to use the new [microservices-based] one. This was done to prevent our client's operation from being interrupted [...] This part of legacy integration had a short lifespan, and new clients had to use the new system.*"

Another challenge practitioners face when defining the microservice architecture is choosing how to approach reuse needs. In another question, we asked whether duplicate code exists in different microservices. For the 15 (28%) participants who said *yes*. Then, we asked them their experiences on handling duplicated code.

Our analysis of this open-ended question revealed two main themes. The first involves using **Shared Libraries and Common Services**. P3 said: "*Building libraries for code reuse,*" and P26 highlighted the reuse of existing services: "*If we identify that the code exists in another microservice, and it makes sense to call that microservice to avoid duplicating the code, reuse is implemented.*". This approach maintains consistency and reduces redundancy.

The second identified theme is **Independent Handling and Lack of Traceability**. P16 explained: "*Every time a change is made, it has to be done in both places and delivered together.*" And P66 shared that duplicate code is "*handled separately in each microservice.*" These are useful insights into why reuse is important in microservices contexts. Such approaches can lead to maintenance challenges, as there is no traceability or centralized management of duplicate code. For instance, P11 stated: "*There is no process or documentation for traceability of duplicate code.*"

Another challenge observed is regarding testing, which is an important aspect of software development. Organizations dedicate a lot of time and effort [49]. Working with microservice architecture adds another level of complexity to that activity [50]. Therefore, participants were questioned about the types of tests performed during the development/migration of microservices in their companies. Our survey revealed that *unit testing* (44 answers, 83%) and *integration testing* (42 answers, 79%) are the most commonly used testing methods. It makes sense that unit tests are preferred since they isolate the smallest usable components of an application before integration. This is particularly effective in microservices because all components are already separated [49]. Also, according to Newman [36], microservices architecture follows standard software engineering practices for tests that happen before deployment.

Integration tests focus on identifying defects in the interactions and interfaces between components [49]. These tests ensure that each component behaves correctly when contacted and that API interfaces deliver the correct information in the appropriate format [49]. This level of testing, however, comes with its own set of microservice-specific challenges. Extensive system tests are impractical due to the higher frequency of

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

releases [51]. Then, companies often rely on fine-grained monitoring techniques in production environments, allowing for the identification and quick correction of failures through new releases [51]. This fits the microservice principle of independently (re)deployable units [36].

Robust monitoring becomes essential to identify potential problems and ensure the ongoing performance and reliability of microservices in production [52]. Thus, we asked participants whether their companies use automatic mechanisms for monitoring systems and what principles of microservice monitoring were used. The responses revealed that a substantial majority (44 answers, 83%) of the practitioners' companies employ automatic mechanisms for monitoring their systems. This monitoring includes checking the performance and scaling of the most utilized microservices, the level of coupling between different microservices, and the use of infrastructure resources. In their study, Mazlami et al. [53] emphasized the importance of robust logging and monitoring tools for achieving a mature level of microservice development. They also noted that 90% of practitioners believe logging and monitoring should be established early in the projects to ensure effective oversight and management of microservice environments.

For the companies that use automatic monitoring, we further investigated the specific principles they apply. The results showed a diverse range of strategies: 36 (68%) *monitor both containers and what runs inside them*, and 36 (68%) *monitor APIs*. Additionally, 27 (51%) utilize *orchestration systems* to manage and monitor their microservices, while 22 (42%) *prepare for elastic and multinational services*. Finally, 12 (23%) *map monitoring to the organizational structure*.

We also questioned participants about the programming languages they used in their microservice-based systems. 25 (47.2%) out of the 53 participants affirmed they used more than one programming language in their projects. This introduces a particular challenge: the need for instrumentation to monitor distributed systems across diverse technology stacks involving various programming languages and paradigms [51]. Furthermore, due to the frequent updates and changes inherent to microservices, defining a "normal" behavior becomes complex, complicating anomaly detection and often leading to false alarms [51]. These challenges highlight the need for robust monitoring strategies to maintain the performance and reliability of microservice environments.

Lastly, the final open question asked participants how their companies manage network overhead. This is an important topic since, unlike monolithic architectures, in microservices-based systems, communication between microservices can introduce significant performance overheads [46].

Our qualitative analysis revealed three main themes regarding challenges. The first theme is **Technical Solutions**, including caching (P10 and P12), load balancing (P10 and P43), and horizontal scaling (P43). Many participants (P15, P21, P25, P27, P39, P72, P74, P78, P79, P81, and P83) rely on cloud infrastructure to manage network overhead, as many current cloud services already offer a solution for this issue. P81 said: "*It mostly comes 'for free' with AWS.*"

In the theme **Monitoring and Design Considerations**, the practitioners P5, P25, and P47 emphasized the importance of monitoring tools, while participants P11 and P67 mentioned changing protocol design and considering network overhead during microservice design.

An interesting theme that emerged in our analysis is that network overhead is **Not a Significant Concern**. Despite existing literature addressing network overhead during the migration [8], [54], for some participants (P4, P8, P13, P28, P66, and P71) the network overhead in microservices' communication is not an issue. Only a few participants noted it as a small concern. We see that in quotes by P20, who explained: "*We just recently started to monitor it [overhead], but it is not a big concern.*" and P25 said that network overhead "*Is not a problem yet, but it is a resource being monitored.*"

Answering RQ4: Few participants (19%) reported reverting to legacy functionalities, and 30% of them dealt with duplicated code. To address this, most participants use shared libraries and reuse existing services. Unit testing (83%) and integration testing (79%) are the most commonly employed for testing. Most of the companies (83%) use automatic monitoring mechanisms. Common strategies include monitoring containers and APIs, using orchestration systems, and mapping monitoring to organizational structures. Network overhead is not a significant concern, with companies leveraging modern cloud infrastructure to ensure efficient communication and scalability across their microservices.

V. Lessons for Practitioners

With our research questions answered, we can extract several practical lessons that can be useful for practitioners, which are presented in this section.

Get your priorities straight: prioritization when migrating is extremely important. Focusing on services with high maintenance or scalability issues can lead to significant early successes. This approach allows organizations to quickly benefit from the improved scalability and faster deployment cycles provided by a microservices architecture. Additionally, starting with less critical services can help teams refine their microservices strategy before dealing with more complex and vital parts of the system.

Knowledge is power: Understanding the existing system is paramount for effective modernization. Leveraging the knowledge of experienced developers and thorough documentation enables a more informed and controlled decomposition of the monolith.

Take small, calculated steps: Incremental decomposition based on business capabilities allows organizations to manage the migration in smaller, less risky steps. Additionally, teams should focus on maintaining a balance between service granularity and complexity; overly granular or overly large services can create management and integration challenges that could negate the advantages of microservices.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

No silver bullets for managing data: Data management in microservices presents its own set of challenges. While adopting a dedicated database per service is ideal for decoupling, it introduces complexity in maintaining data consistency. Strategies such as eventual consistency and atomic transactions provide different benefits depending on the system's requirements. Organizations should prioritize asynchronous processing, event-driven architectures, and continuous monitoring to manage scaling effectively. This approach, however, requires careful consideration of legacy dependencies, cost constraints, and the difficulties of database partitioning.

Address code duplication proactively: Practitioners should be aware of the challenges of managing duplicated code, which can arise during modernization. Companies often address this by utilizing shared libraries and reusing existing services, which helps maintain consistency and reduce redundancy across the system.

Testing is not an afterthought: The complexities of testing in a microservices environment require a strategic approach. Our participants employ a mix of unit and integration testing to ensure the system's reliability. This helps to validate individual services and their interactions, ensuring that the system functions as intended.

Monitor everything, automate everywhere: Robust monitoring, primarily through automated mechanisms, is essential for maintaining the health of microservices. Effective strategies include monitoring containers and APIs, using orchestration systems, and aligning monitoring practices with organizational structures to ensure comprehensive oversight.

Leverage the cloud for network challenges: While network overhead might seem a theoretical concern, most organizations find it manageable with modern cloud infrastructure. Cloud services offer built-in solutions for scalability and performance, highlighting the importance of a proactive monitoring approach and strategic use of cloud resources to mitigate performance challenges in microservice environments.

VI. Threats to Validity

In this section, we present the threats to the validity of our work and their mitigation [55].

Internal Validity: One potential threat is the diversity of participants and companies. Although our sample includes responses from various international practitioners, it may not represent a regular distribution across all regions. This diversity is needed for our study but also introduces variability in practices and experiences that may affect the results. However, our sample covers a range of companies with different sizes and domains, with practitioners based in different countries.

External Validity: Although diverse, we cannot claim that our survey sample represents all types of companies, considering factors such as size, domain, and geographical location, impacting the generalization of our results. To mitigate this threat, we distributed the questionnaire through various media and social networks to reach a broad audience.

Construct Validity: Several survey questions were close-ended with predefined responses, which might have led participants to choose these options instead of describing their specific situations. To address this, we validated them in a pilot study with experienced developers. The use of a higher number of close-ended questions aimed to reduce the complexity and time required to complete the survey, thereby minimizing participant fatigue.

Conclusion Validity: The conclusions drawn from our data rely on the accurate and honest reporting of experiences by participants. Any misreporting or misunderstanding of survey questions could affect the validity of our conclusions. We designed our survey to be clear and concise, conducting a pilot study to refine questions. Nevertheless, the self-reported nature of the data poses an inherent limitation.

VII. Conclusion

This paper explores the motivations, activities, and challenges associated with migrating monolithic legacy systems to microservices. Our research aimed to understand how professionals conduct their migrations and handle the associated challenges, providing insights from 53 software practitioners across diverse international contexts.

Our main findings reveal that the primary driving forces for adopting microservices are operational and technical, focusing on enhancing maintenance, scalability, and deployment processes. Unit and integration testing, alongside extensive monitoring, are essential for managing dynamic microservice environments. Database management continues to be an issue, with companies divided between centralized and decentralized approaches. Additionally, Modern cloud technologies are used to mitigate network overhead in microservice architectures.

As contributions, our study provides a comprehensive view of the current practices and challenges in migrating to microservices from the perspective of industry. The diverse approaches and ongoing challenges reveal the need for continued research and tailored solutions to support successful microservice adoption.

Acknowledgment

This work is supported by CNPq grant 404027/2023-7, FAPERJ PDR-10 program 202073/2020, and CAPES - Finance Code 001.

References

[1] R. C. Seacord, D. Plakosh, and G. A. Lewis, *Modernizing Legacy Systems: Software Technologies, Engineering Process and Business Practices*. Addison-Wesley Longman Publishing Co., Inc., 2003.

[2] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, "Decomposition of monolith applications into microservices architectures: A systematic review," *IEEE Transactions on Software Engineering*, 2023.

[3] A. Stojkov and Z. Stojanov, "Review of methods for migrating software systems to microservices architecture," *Journal of Engineering Management and Competitiveness*, vol. 11, pp. 152–162, 01 2021.

[4] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, *Microservices: Yesterday, Today, and Tomorrow*, pp. 195–216. Cham: Springer International Publishing, 2017.

[5] P. Jamshidi, A. Ahmad, and C. Pahl, "Cloud migration research: A systematic review," *IEEE Transactions on Cloud Computing*, vol. 1, no. 2, pp. 142–157, 2013.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

[6] R. Chen, S. Li, and Z. Li, "From monolith to microservices: A dataflow-driven approach," in *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 466–475, 2017.

[7] D. Taibi, V. Lenarduzzi, and C. Pahl, "Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation," *IEEE Cloud Computing*, vol. 4, no. 5, pp. 22–32, 2017.

[8] L. Carvalho, T. E. Colanzi, W. K. G. Assunção, A. Garcia, J. A. Pereira, M. Kalinowski, R. M. de Mello, M. J. de Lima, and C. Lucena, "On the usefulness of automatically generated microservice architectures," *IEEE Transactions on Software Engineering*, vol. 50, no. 3, pp. 651–667, 2024.

[9] J. Lewis and M. Fowler, "Microservices: a definition of this new architectural term," 2014.

[10] T. Erl, *Service-Oriented Architecture: Concepts, Technology, and Design*. Prentice Hall e, 2005.

[11] A. Henry and Y. Ridene, *Migrating to Microservices*, pp. 45–72. Cham: Springer International Publishing, 2020.

[12] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices architecture enables devops: Migration to a cloud-native architecture," *IEEE Software*, vol. 33, no. 3, pp. 42–52, 2016.

[13] D. Wolfart, W. K. G. Assunção, I. F. da Silva, D. C. P. Domingos, E. Schmeing, G. L. D. Villaca, and D. d. N. Paza, "Modernizing legacy systems with microservices: A roadmap," in *Evaluation and Assessment in Software Engineering*, EASE 2021, (New York, NY, USA), p. 149–159, Association for Computing Machinery, 2021.

[14] A. Bucchiarone, N. Dragoni, S. Dustdar, S. T. Larsen, and M. Mazzara, "From monolithic to microservices: An experience report from the banking domain," *IEEE Software*, vol. 35, no. 3, pp. 50–55, 2018.

[15] P. Cruz, H. Astudillo, R. Hilliard, and M. Collado, "Assessing migration of a 20-year-old system to a micro-service platform using atam," in *2019 IEEE International Conference on Software Architecture Companion (ICSA-C)*, pp. 174–181, 2019.

[16] H. E. Hayretci and F. B. Aydemir, "A multi case study on legacy system migration in the banking industry," in *Advanced Information Systems Engineering* (M. La Rosa, S. Sadiq, and E. Teniente, eds.), (Cham), pp. 536–550, Springer International Publishing, 2021.

[17] X. Zhou, S. Li, L. Cao, H. Zhang, Z. Jia, C. Zhong, Z. Shan, and M. A. Babar, "Revisiting the practices and pains of microservice architecture in reality: An industrial inquiry," *Journal of Systems and Software*, vol. 195, p. 111521, 2023.

[18] J. Fritzsch, J. Bogner, S. Wagner, and A. Zimmermann, "Microservices migration in industry: Intentions, strategies, and challenges," in *35th Intl. Conference on Software Maintenance and Evolution (ICSME)*, pp. 481–490, 2019.

[19] P. Di Francesco, P. Lago, and I. Malavolta, "Migrating towards microservice architectures: An industrial survey," in *2018 IEEE International Conference on Software Architecture (ICSA)*, pp. 29–2909, 2018.

[20] J. Fritzsch, J. Bogner, A. Zimmermann, and S. Wagner, "From monolith to microservices: a classification of refactoring approaches," in *1st Intl. Workshop on Software Engineering Aspects of Continuous Development and New Paradigms of Software Production and Deployment (DEVOPS)*, pp. 128–141, 2018.

[21] T. Colanzi, A. Amaral, W. Assunção, A. Zavadski, D. Tanno, A. Garcia, and C. Lucena, "Are we speaking the industry language? the practice and literature of modernizing legacy systems with microservices," in *15th Brazilian Symposium on Software Components, Architectures, and Reuse*, SBCARS '21, (New York, NY, USA), p. 61–70, Association for Computing Machinery, 2021.

[22] Y. Wang, H. Kadiyala, and J. Rubin, "Promises and challenges of microservices: an exploratory study," *Empirical Software Engineering*, vol. 26, no. 4, pp. 1–44, 2021.

[23] V. Lopes Nogueira, "Microservice migration survey and responses." https://doi.org/10.5281/zenodo.13324295, June 2024.

[24] D. S. Cruzes and T. Dyba, "Recommended steps for thematic synthesis in software engineering," in *2011 International Symposium on Empirical Software Engineering and Measurement*, pp. 275–284, 2011.

[25] L. Carvalho, A. Garcia, W. K. G. Assunção, R. de Mello, and M. J. de Lima, "Analysis of the criteria adopted in industry to extract microservices," in *7th Intl. Workshop on Conducting Empirical Studies in Industry / 6th Intl. Workshop on Software Engineering Research and Industrial Practice (CESSER-IP)*, pp. 22–29, 2019.

[26] R. Chen, S. Li, and Z. Li, "From Monolith to Microservices: A Dataflow-Driven Approach," in *25th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 466–475, 2018.

[27] M. Fowler, "Domain driven design," 2020.

[28] V. Velepucha and P. Flores, "A survey on microservices architecture: Principles, patterns and migration challenges," *IEEE Access*, vol. 11, pp. 88339–88358, 2023.

[29] E. Aghajani, C. Nagy, M. Linares-Vásquez, L. Moreno, G. Bavota, M. Lanza, and D. C. Shepherd, "Software documentation: the practitioners' perspective," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, ICSE '20, (New York, NY, USA), p. 590–601, Association for Computing Machinery, 2020.

[30] A. Sampaio Junior, J. Rubin, I. Beschastnikh, and N. Rosa, "Improving microservice-based applications with runtime placement adaptation," *Journal of Internet Services and Applications*, vol. 10, 12 2019.

[31] A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Migrating to cloud-native architectures using microservices: An experience report," in *Advances in Service-Oriented and Cloud Computing* (A. Celesti and P. Leitner, eds.), (Cham), pp. 201–215, Springer International Publishing, 2016.

[32] R. M. Meloca, R. Ré, and A. L. Schwerz, "An analysis of frameworks for microservices," *Latin American Computer Conf.*, pp. 542–551, 2018.

[33] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.

[34] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study in microservice architecture," in *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, pp. 44–51, 2016.

[35] C.-Y. Fan and S.-P. Ma, "Migrating monolithic mobile application to microservice architecture: An experiment report," in *2017 IEEE International Conference on AI & Mobile Services (AIMS)*, pp. 109–112, 2017.

[36] S. Newman, *Building microservices: designing fine-grained systems*. O'Reilly, 2015.

[37] L. J. Kirby, E. Boerstra, Z. J. Anderson, and J. Rubin, "Weighing the evidence: On relationship types in microservice extraction," in *2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC)*, pp. 358–368, 2021.

[38] R. Mishra, N. Jaiswal, R. Prakash, and P. N. Barwal, "Transition from monolithic to microservices architecture: Need and proposed pipeline," in *2022 International Conference on Futuristic Technologies (INCOFT)*, pp. 1–6, 2022.

[39] M. Wu, Y. Zhang, J. Liu, S. Wang, Z. Zhang, X. Xia§, and X. Mao, "On the way to microservices: Exploring problems and solutions from online qa community," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 432–443, 2022.

[40] V. Velepucha and P. Flores, "Monoliths to microservices - migration problems and challenges: A sms," in *2021 Second International Conference on Information Systems and Software Technologies (ICI2ST)*, pp. 135–142, 2021.

[41] V. N. Saša Baškarada and A. Koronios, "Architecting microservices: Practical opportunities and challenges," *Journal of Computer Information Systems*, vol. 60, no. 5, pp. 428–436, 2020.

[42] N. A. N. Sobri, M. A. H. Abas, I. M. Yassin, M. S. A. M. Ali, N. M. Tahir, A. Zabidi, and Z. I. Rizman, "A study of database connection pool in microservice architecture," *JOIV : International Journal on Informatics Visualization*, 2022.

[43] D. Faustino, N. Gonçalves, M. Portela, and A. Rito Silva, "Stepwise migration of a monolith to a microservice architecture: Performance and migration effort evaluation," *Performance Evaluation*, vol. 164, p. 102411, 2024.

[44] E. Ntentos, U. Zdun, K. Plakidas, D. Schall, F. Li, and S. Meixner, "Supporting architectural decision making on data management in microservice architectures," in *Software Architecture* (T. Bures, L. Duchien, and P. Inverardi, eds.), (Cham), pp. 20–36, Springer International Publishing, 2019.

[45] C. Richardson, "Microservices - pattern: Database per service," 2016.

[46] M. Kalske, N. Mäkitalo, and T. Mikkonen, "Challenges when moving from monolith to microservice architecture," in *Current Trends in Web Engineering* (I. Garrigós and M. Wimmer, eds.), (Cham), pp. 32–47, Springer International Publishing, 2018.

[47] M. Fowler, "Cqrs," 2011.

[48] J. Vučković, *You Are Not Netflix*, pp. 333–346. Cham: Springer International Publishing, 2020.

[49] I. Ghani, W. M. Wan-Kadir, A. Mustafa, and M. I. Babir, "Microservice testing approaches: A systematic literature review," *International Journal of Integrated Engineering*, vol. 11, no. 8, pp. 65–80, 2019.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.

[50] C. Richardson and F. Smith, *Microservices - From Design to Deployment*. "NGINX, Inc.", 2016.
[51] R. Heinrich, A. van Hoorn, H. Knoche, F. Li, L. E. Lwakatare, C. Pahl, S. Schulte, and J. Wettinger, "Performance engineering for microservices: Research challenges and directions," in *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, ICPE '17 Companion, (New York, NY, USA), p. 223–226, Association for Computing Machinery, 2017.
[52] M. Waseem, P. Liang, M. Shahin, A. Di Salle, and G. Márquez, "Design, monitoring, and testing of microservices systems: The practitioners' perspective," *Journal of Systems and Software*, vol. 182, p. 111061, 2021.
[53] G. Mazlami, J. Cito, and P. Leitner, "Extraction of microservices from monolithic software architectures," in *24th Intl. Conf. on Web Services (ICWS)*, pp. 1–8, 2017.
[54] L. Carvalho, A. Garcia, T. E. Colanzi, W. K. G. Assunção, J. A. Pereira, B. Fonseca, M. Ribeiro, M. J. de Lima, and C. Lucena, "On the performance and adoption of search-based microservice identification with tomicroservices," in *36th Intl. Conference on Software Maintenance and Evolution (ICSME)*, pp. 569–580, 2020.
[55] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering: An Introduction*. Kluwer Academic Publishers, 2000.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 09:40:25 UTC from IEEE Xplore. Restrictions apply.